COSC 461/561 Project 1: Eagle Delivery

Ahmed Nowayti

Part A: A* Search

(a) Formulation. State: the robot's (row, column). Initial state: the start building. Actions: UP, DOWN, LEFT, RIGHT. Transition: move one cell, or stay in place if it is a wall or outside the map. Goal test: the robot is at the destination. Path cost: 1 per move. BFS is optimal because every move costs 1, and BFS checks shorter routes before longer ones, so the first route it finds is the shortest.

(b) Manhattan distance is |row difference| + |column difference|. It is admissible because each move changes the row or column by only 1, and walls can only make the route longer, so it never overestimates.

(c) A*. I used `heapq` with entries (f, insertion number, g, state). I skip old entries, test for the goal when popping, and count a node when I generate its neighbors.

Map	Task	BFS length	A* length	BFS expanded	A* expanded
Baseline	S -> W	18	18	67	22
Baseline	L -> D	22	22	79	62
Baseline	H -> P	16	16	41	16
Detour	S -> W	26	26	70	51
Detour	L -> D	22	22	65	60
Detour	H -> P	16	16	41	16

Both found routes of the same length, since both are optimal. A* expanded fewer nodes every time, but the savings were different. For H -> P the route goes almost straight to P, so A* only expanded 16 nodes. For L -> D the walls make the robot go away from D first, so the heuristic did not help much.

Part B: Q-Learning

`epsilon_greedy` picks a random action with probability epsilon, otherwise the action with the highest Q-value (ties broken randomly). `q_learning_update` uses Q = Q + alpha × (target − Q). Settings: 2000 training and 100 evaluation episodes, seeds 0, 1, 2, alpha = 0.3, gamma = 0.95, epsilon = 0.2.

Task	Eval success rate	Eval steps (BFS shortest)	Avg training steps, first 100 episodes	Avg training steps, last 100 episodes
S -> W	100%	18 (18)	98	23
L -> D	100%	22 (22)	94	28
H -> P	100%	16 (16)	58	20

- All seeds delivered every time in evaluation, using the shortest route.
- The agent learned fast: after about 200 episodes it succeeded almost every time.
- Training episodes stayed a bit longer than the shortest route because 20% of actions are random. In evaluation epsilon is 0, so this goes away.



Average training steps per episode (3 seeds).

Part C: Custom Map and Experiment

(a) My map (`maps/my_campus.txt`, 12 x 19):

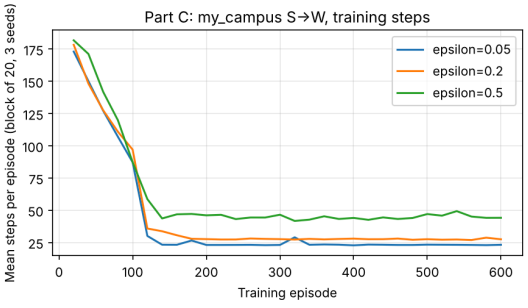
```
###################
#S.....#........W#
#.#####.#.#######.#
#.#...#.#.#.....#.#
#.#.#.#...#.###.#.#
#...#.#####.#H#...#
###.#.......#.#####
#L#.#####.###.....#
#.#.....#...#.###.#
#.#####.###.#.#D#.#
#.......#P....#...#
###################
```

It has one loop through S and W: a short top route (22 steps) and a longer middle route (34 steps). It also has dead-end branches leading to L, and to P, D, and H. On S -> W, BFS found length 22 with 65 expanded and A* found length 22 with 25 expanded.

(b) Question and prediction. How does epsilon affect learning? I predicted a higher epsilon would find the short route more reliably but have longer training episodes, and that epsilon = 0.05 might get stuck on the long route.

(c) Method. I tested epsilon = 0.05, 0.2, 0.5 on the S -> W task, with seeds 0, 1, 2 each. Everything else was the same as Part B.

Epsilon	Eval success rate	Eval steps	Avg training steps, last 100 episodes
0.05	100%	22	23
0.2	100%	22	27
0.5	100%	22	44



Average training steps per episode on my map (3 seeds).

(d) Discussion. Epsilon made no difference to the final result: every setting learned the short 22-step route. My guess that 0.05 would get stuck was wrong. I think this is because every step costs −1 and untried actions start at 0, so the agent tries new actions anyway. Epsilon did matter during training: with 0.5, episodes stayed around 44 steps because half the moves were random, while 0.05 stayed close to 22. So on this small map, a small epsilon was better.